

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool (BL3)

Assessment Tool Weightage: 20%

Duration :60 Mins

On class

Total Marks:50

QUESTIONS:

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

1. a) Error Identification:

Line	Error	Reason
9	int 2num = 10;	identifier can't start with digit
10	Missing ;	Syntax Error
12	square();	Function argument missing
13	if (result == 100)	Assignment used instead of (==)
15	Missing ; after printf	Syntax Error
17	Total undeclared	Semantic Error
18	arr[5]	Array index out of bounds
20	*p = 50;	Uninitialized pointer
22	x/y (y=0)	Division by zero
24-27	while (i <= 5)	Infinite Loop (i++ missing).

b) Classification:

Error	Phase
Invalid Identifier	Lexical
Missing Semicolons	Syntax
Missing function argument	Semantic
Undeclared variable	Semantic
Assignment in Condition	Logical
Array out of bounds	Runtime
Uninitialized pointer	Runtime
Division by zero	Runtime
Infinite Loop	Logical

c) Compilation Behaviour:

1. **Lexical** - Invalid identifier (num)
2. **Syntax** - Missing semicolons
3. **Semantic** - Missing function argument, undeclared variable.
4. **Runtime** - Division by zero, invalid pointer, array out of bounds.
5. Earlier Lexical / Syntax errors may stop the compiler before detecting later errors.

d) Code Corrections:

```
int num = 10;
Add missing;
result = square(num);
if (result == 100)
Declare int total;
Use arr[4]
int value = 50; int *p = &value;
int y = 2;
Add i++; inside the loop.
```

e) Impact Analysis:

Uninitialized Pointer: May cause segmentation fault.

Division by zero: Program crashes or gives undefined result.

Array out of bounds: Accesses invalid memory.

Infinite Loop: Program never terminates.

Assignment (=) in if: Produces incorrect logic.

f) Advanced Compiler Perspectives:

1. Generate clearer error messages.
2. Warn about division by zero.
3. Detect array boundary violations.
4. Detect uninitialized variables / pointers.

a) Error Identification:

Line	Error	Reason
4	int a = 10	Missing Semicolon (;).
6	if (a < b	Missing closing parenthesis).
10	for (int i = 0; i < 5; i++	Missing closing parenthesis) before {.
14	while (a < 20	Missing closing parenthesis).
17	printf ("Sum = %.d\n", a + b;	Missing closing parenthesis)

b) Error Classification:

Error	Category
Missing Semicolon	Missing Statement terminator
Missing) in if	Unmatched parentheses.
Missing) in for	Incorrect control structure.
Missing) in while	Incorrect control structure.
Missing) in printf	Unmatched Parenthesis.

c) Syntax Correction:

```

int a = 10;
if (a < b)
for (int i = 0; i < 5; i++)
while (a < 20)
printf ("Sum = %.d\n", a + b);

```

d) Compiler Behavior Analysis:

1. **Parser** checks whether the program follows C grammar using CFG.
2. It detects missing semicolons, braces & parentheses during syntax analysis.

3. The parser may stop after the 1st error, but error recovery techniques allow it to continue & report multiple errors.

e) Advanced Thinking:

Grammar Rule:

if-statement $\rightarrow$ if (condition) statement.

Missing) Detection

The parser expects a closing) after the condition. If it is missing, it reports a syntax error.

Two Parser Improvement:

1. Display clear, user-friendly error messages with line numbers.
2. Suggest possible fixes (e.g: "Did you forget a ')' or ';' ?").

3. a) Identify Errors:

Line	Error	Reason
7	int # count = 5;	Invalid identifier (# not allowed).
8	float rate = 2.5	Missing semicolon (;).
10	result = square();	Missing function argument.
11-15	if ... else	Missing closing brace before else.
14	int z = x / y;	Division by zero (y = 0).
21	marks[4]	Array index out of bounds (valid: 0-3).
23	*ptr = 25;	Uninitialized pointer.
24	total = x + 0 + y;	Redundant arithmetic (+0).
26-27	while (i <= 5)	Infinite loop (i++ missing).

b) Classification:

Error	Compiler Phase
Invalid Identifier (#count)	Lexical
Missing semicolon	Syntax
Missing function argument	Semantic
Missing brace in if-else.	Syntax
Division by zero.	Runtime
Array out of bounds	Runtime
Uninitialized pointer	Runtime
$x + 0 + y$	Code Optimization
infinite loop	Runtime / Logical

c) Compilation vs Execution:

1. **Lexical** - Invalid identifier (#count).
2. **Syntax** - Missing semicolon & missing brace.
3. **Semantic** - Incorrect function call (square()).
4. **Runtime** - Division by zero, array out of bounds, uninitialized pointer, infinite loop.
5. Lexical and Syntax errors must be fixed before later compiler phases proceed.

d) Program Correction:

```
Change #count -> count
Add missing ;
Use result = square(count);
Add missing { before else
Set y = 2
Access marks [3]
Initialize pointer:
int value = 25;
int *ptr = &value;
```

Simplify:

$total = x + y;$

add $i++$; inside the while loop.

e) Deep Analysis:

1. Uninitialized pointer: May cause segmentation fault.
2. Array out of bounds: accesses invalid memory.
3. Infinite Loop: Program never terminates.
4. Division by zero: Causes runtime error.
5. Missing function argument: Function call becomes invalid.

Effects:

1. Reduces memory safety.
2. Produces incorrect results.
3. Can crash the system.

f) Advanced Compiler Perspective:

1. Detect uninitialized pointers at compile time.
2. Warn about possible division by zero.
3. Check array boundary violations.
4. Detects possible infinite loops using static analysis.
5. Optimize $total = x + 0 + y;$ to $total = x + y;$ for more efficient code.